Instituto Tecnológico de Mazatlán

Botón de arranque/paro a prueba de rebotes y apagones con Arduino UNO R4 WiFi

Carrera:
Ingeniería en Sistemas Computacionales

Materia:
Sistemas Programables

Profesor:
Miguel Barrón Hernández

Integrantes:
Ximena Armenta Dávila
Carlos Tadeo Ibarra Castañeda
Dulce Princesa Ulivarría González

1. Introducción

Al programar solemos suponer que el voltaje siempre es de 5 V y que un botón cambia de estado una sola vez cada vez que se oprime, pero en el mundo físico ninguna de las dos cosas es cierta. En esta práctica un solo botón arranca y detiene un motor, y el programa debe defenderse de dos fenómenos reales: el rebote de los contactos del botón y las caídas de voltaje que reinician al Arduino.

El rebote ocurre porque la lámina metálica del botón choca contra los contactos y rebota durante 1 a 10 ms antes de quedarse quieta. Una persona no lo nota, pero el Arduino es tan rápido que ve varias pulsaciones donde hubo una sola, y el motor a veces "no hace caso". La caída de voltaje (brownout) es una baja momentánea de la alimentación provocada, por ejemplo, por el arranque del motor, una fuente débil o cables flojos. Si el voltaje baja del mínimo, el microcontrolador se reinicia y puede hacer algo que nadie ordenó, como arrancar un motor por sí solo.

Para el rebote se programó un filtro no bloqueante dentro de una clase en C++ llamada `Boton`, que recuerda la última lectura, el estado ya aceptado, el momento del último cambio y una ventana de estabilidad de 25 ms, y avisa una sola vez por cada pulsación real (cuando el botón pasa de suelto a oprimido). Para el apagón, `setup()` deja el motor apagado, se muestra un aviso de "sistema reiniciado" en la pantalla y en el Monitor Serie, el motor arranca con una rampa de PWM de 0 a 255 en aproximadamente 1.5 s y, después de cualquier reinicio, solo vuelve a arrancar si alguien oprime el botón. Todo se hizo con `millis()` y sin usar `delay()`.

La práctica se armó y probó en el simulador Tinkercad, donde el puente H es un L293D, la fuente del motor es una pila de 9 V y, en lugar de la matriz LED del Arduino UNO R4 WiFi, se usó una pantalla LCD I2C para mostrar el estado del motor.

2. Capturas del circuito funcionando

En la Figura 1 se observa la simulación en ejecución: el motor está apagado, la pantalla muestra "Motor: OFF" y el Monitor Serie registra las pulsaciones #2, #3 y #4, el cambio de la ventana del filtro (300 ms y luego 25 ms) y el arranque con rampa de 1.5 s. En la Figura 2 se muestra la prueba del apagón: después de reiniciar con el motor a velocidad máxima, la pantalla y el Monitor Serie muestran el aviso de sistema reiniciado y el motor queda apagado.

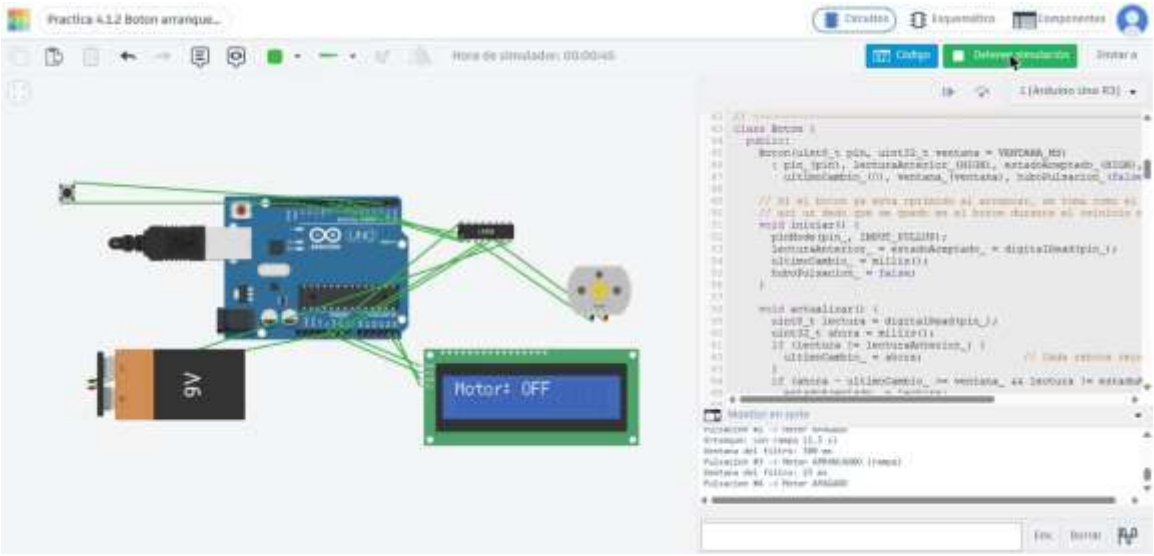


Figura 1. Simulación en Tinkercad con la pantalla LCD (Motor: OFF) y el Monitor Serie con las pulsaciones contadas, el cambio de ventana y el arranque con rampa.

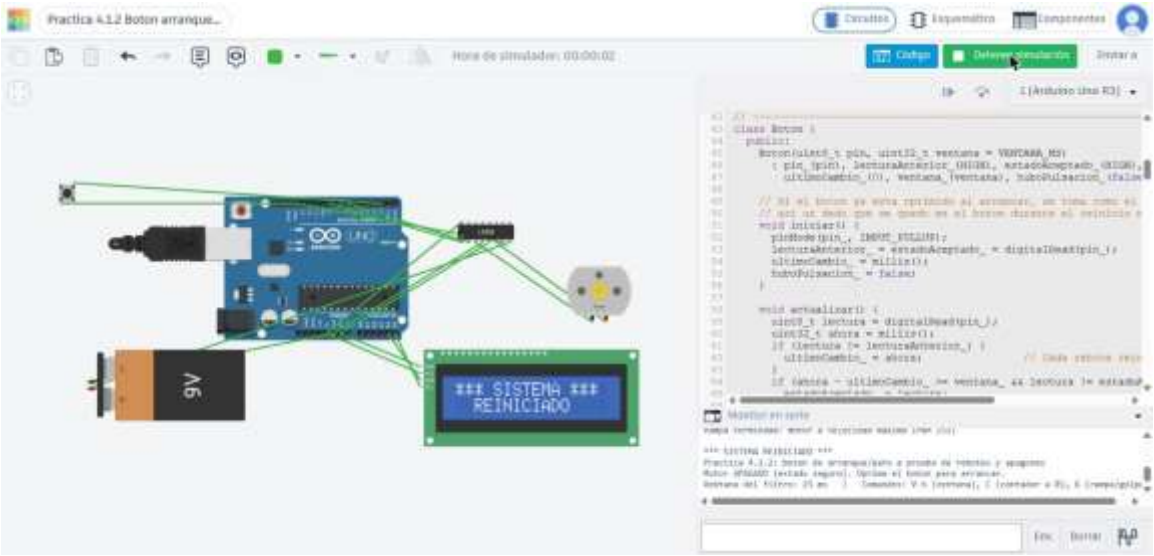


*Figura 2. Prueba del apagón: tras el reinicio aparece "*** SISTEMA REINICIADO ***" en la pantalla y en el Monitor Serie, y el motor queda en estado seguro (apagado).*

3. Metodología

Primero se armó el circuito siguiendo las conexiones de la guía de la práctica, separando la parte de potencia de la de señal. En la potencia, el positivo del portapilas va a la terminal +12V del L298N, el negativo a su terminal GND y el motorreductor a las salidas OUT1 y OUT2. El GND del L298N se unió al GND del Arduino para tener una tierra común, que es obligatoria; se dejó puesto el jumper de 5V-EN y se quitó el jumper de ENA para poder controlar la velocidad con PWM. En la señal, D9 se conectó a ENA (velocidad con PWM para la rampa), D7 a IN1 (encendido) e IN2 se llevó a GND para fijar el sentido de giro. El botón va de D2 a GND y no lleva resistencia externa, porque se usa la resistencia interna del Arduino con `INPUT_PULLUP`.

Se tuvo cuidado de no alimentar el motor desde el pin 5V del Arduino, de no conectar la terminal +5V del L298N al Arduino y de revisar la polaridad del portapilas antes de conectar.

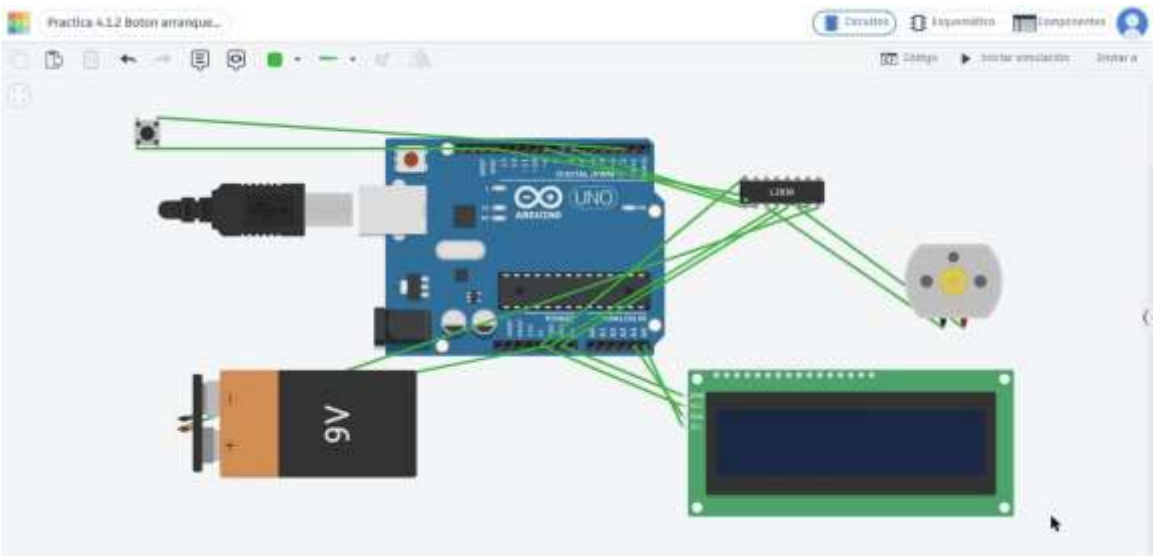


Figura 3. Circuito simulado en Tinkercad (en la simulación el puente H es un L293D, la fuente del motor una pila de 9 V y la matriz LED se sustituye por una pantalla LCD I2C).

Tabla I. Conexiones del circuito con el Arduino UNO R4 WiFi.

Elemento / pin	Conexión	Función
Portapilas (+)	+12V del L298N	Alimentación de potencia del motor.
Portapilas (−)	GND del L298N	Tierra de la parte de potencia.
GND del L298N	GND del Arduino	Tierra común, obligatoria.
Motorreductor	OUT1 y OUT2 del L298N	Salida de potencia al motor.
ENA del L298N	D9	Velocidad del motor con PWM (rampa).
IN1 del L298N	D7	Orden de encendido del motor.
IN2 del L298N	GND	Fija el sentido de giro.
Botón (pata 1)	D2	Entrada con INPUT_PULLUP (suelto = HIGH, oprimido = LOW).
Botón (pata 2)	GND	Referencia del botón, sin resistencia externa.
Pantalla LCD I2C (solo simulación)	5V, GND, SDA y SCL	Muestra el estado del motor en lugar de la matriz LED.

Antes de programar se dibujó el diagrama de estados del sistema (Figura 4). Al reiniciar, el sistema siempre entra en APAGADO. Una pulsación real, ya filtrada, pasa a ARRANCANDO, donde el PWM sube de 0 a 255 en aproximadamente 1.5 s, y al terminar la rampa el motor queda ENCENDIDO a velocidad máxima. Otra pulsación apaga el motor de inmediato (PWM en 0), y una caída de voltaje o un RESET regresa el sistema a APAGADO con el aviso de reinicio.



Figura 4. Diagrama de estados del sistema con el estado seguro inicial, la rampa de arranque y el regreso a APAGADO tras un reinicio.

El programa se organiza de la siguiente manera:

- **Datos privados de la clase `Boton`:** el pin (D2), la lectura anterior (HIGH), el estado aceptado (HIGH), el momento del último cambio medido con `millis()` (0), la ventana de estabilidad (25 ms) y la bandera `huboPulsacion_` (falso).
- **`iniciar()`:** configura el pin con `INPUT_PULLUP` y toma la lectura actual como estado aceptado. Así, si el botón ya estaba oprimido al arrancar (por ejemplo, un dedo que se quedó en él durante el reinicio), no se cuenta como una pulsación nueva y el motor no arranca solo.
- **`actualizar()`:** se llama en cada vuelta del `loop()`. Lee el pin y, si la lectura cambió, guarda la hora actual; cada rebote reinicia ese cronómetro. Solo cuando la señal lleva 25 ms o más sin cambiar y es distinta del estado aceptado, se acepta el nuevo estado, y si es LOW (oprimido) se enciende la bandera `huboPulsacion_`. Al final guarda la lectura como lectura anterior.
- **`fuePresionado()`:** si la bandera está encendida, la apaga y regresa verdadero; si no, regresa falso. Así cada pulsación real se entrega una sola vez y el soltar el botón nunca cuenta.
- **Estado seguro:** lo primero que hace `setup()` es dejar el motor apagado (PWM en 0 e IN1 en LOW), sin importar cómo estuviera antes del reinicio, y después muestra el aviso de "sistema reiniciado" en la pantalla y en el Monitor Serie. El motor solo arranca con una pulsación nueva.
- **Rampa de arranque:** el PWM sube de 0 a 255 en aproximadamente 1.5 s, en pasos medidos con `millis()`. El paro es inmediato (PWM a 0). La pantalla muestra el estado del motor mientras el sistema funciona.
- **Comandos por el Monitor Serie:** para las pruebas se agregaron los comandos de la Tabla II, que permiten cambiar la ventana del filtro, reiniciar el contador y alternar el tipo de arranque sin volver a cargar el programa.
- **Sin `delay()`:** toda la espera del filtro y de la rampa se mide con `millis()`, por lo que el `loop()` nunca se bloquea.

```
void actualizar() {
  uint8_t lectura = digitalRead(pin_);
  uint32_t ahora = millis();
  if (lectura != lecturaAnterior_) {
    ultimoCambio_ = ahora;            // cada rebote reinicia el cronometro
  }
  if (ahora - ultimoCambio_ >= ventana_ && lectura != estadoAceptado_) {
    estadoAceptado_ = lectura;        // la senal se quedo quieta: se acepta
    if (estadoAceptado_ == LOW) huboPulsacion_ = true;  // solo al oprimir
  }
  lecturaAnterior_ = lectura;
}
```

Tabla II. Comandos del Monitor Serie.

Comando	Acción
V n	Cambia la ventana del filtro a n milisegundos (por ejemplo V 2, V 25 o V 300).
C	Pone el contador de pulsaciones en cero.
G	Alterna entre el arranque con rampa y el arranque de golpe (PWM directo a 255).

4. Análisis de resultados

Respuesta del sistema

La Tabla III resume lo que se observó en la pantalla y en el Monitor Serie. Con el filtro de 25 ms cada pulsación real aparece una sola vez en el contador, y el soltar el botón no genera ningún mensaje.

Tabla III. Eventos probados y respuestas observadas.

Evento	Respuesta observada	Interpretación
Arranque o RESET	“*** SISTEMA REINICIADO ***” en la pantalla y en el Monitor Serie, y “Motor APAGADO (estado seguro). Oprima el boton para arrancar.”	Estado seguro: el motor queda apagado y se avisa del reinicio.
Pulsación con el motor apagado	“Pulsacion #n -> Motor ARRANCANDO (rampa)” y después “Rampa terminada: motor a velocidad maxima (PWM 255)”	El PWM sube poco a poco de 0 a 255 en 1.5 s.
Pulsación con el motor encendido	“Pulsacion #n -> Motor APAGADO” y la pantalla muestra “Motor: OFF”	Paro inmediato: PWM en 0.
V 300 / V 25	“Ventana del filtro: 300 ms” / “Ventana del filtro: 25 ms”	Se cambia la ventana sin recargar el programa.
G	“Arranque: con rampa (1.5 s)”	Se elige el tipo de arranque.

Prueba de rebote y experimento con la ventana

La prueba consiste en oprimir el botón 10 veces, despacio y una a la vez, soltándolo por completo entre una y otra, y anotar cuántas pulsaciones cuenta el Arduino y cuántas veces el motor no obedeció. Como el pulsador de Tinkercad es ideal y no rebota, los valores de la Tabla IV se obtuvieron con una simulación

por software del rebote: se generó la señal cruda del botón para 10 pulsaciones lentas (con tramos de rebote de 1 a 10 ms al oprimir y al soltar, y tiempos de pulsación de 120 a 450 ms) y se le aplicó tanto el contador sin filtro como el algoritmo de la clase `Boton` con cada ventana. Se considera que el motor no obedeció cuando, en una pulsación, el número de conmutaciones detectadas es par o cero y el motor queda como estaba. Las pruebas de las ventanas se hicieron con la misma señal para poder compararlas.

Tabla IV. Resultados simulados de la prueba de rebote (10 pulsaciones por prueba).

Prueba	Pulsaciones reales	Pulsaciones contadas	Veces que el motor no obedeció
Sin filtro, intento 1	10	13	3
Sin filtro, intento 2	10	11	1
Sin filtro, intento 3	10	13	3
Ventana 2 ms	10	11	1
Ventana 25 ms	10	10	0
Ventana 300 ms	10	5	5

- **Sin filtro:** el contador suma cada paso de HIGH a LOW que ve el Arduino, así que cualquier rebote al oprimir, o la bajada falsa que aparece al soltar, cuenta como una pulsación de más. En los tres intentos contó 13, 11 y 13 pulsaciones en lugar de 10, y el motor no obedeció 3, 1 y 3 veces, porque con un número par de pulsaciones detectadas en una sola pulsación real el motor queda como estaba.
- **Ventana de 2 ms:** es más corta que los rebotes típicos (1 a 10 ms), por lo que algunos rebotes se "colaron": contó 11 pulsaciones en lugar de 10 y el motor no obedeció 1 vez.
- **Ventana de 25 ms:** es mayor que los rebotes y menor que lo que tarda una persona en oprimir y soltar, por lo que contó exactamente 10 de 10 y el motor obedeció todas las veces; el botón se siente inmediato.
- **Ventana de 300 ms:** los rebotes se filtran, pero el botón se siente lento porque la pulsación se acepta hasta que la señal lleva 300 ms quieta, y las pulsaciones más cortas que ese tiempo se pierden: contó solo 5 de 10 y el motor no obedeció 5 veces.
- **Nota sobre los datos:** los valores de la Tabla IV son resultados de una simulación por software del rebote, no mediciones con un botón físico, porque el pulsador de Tinkercad no rebota. En el simulador sí se comprobó que el filtro y los comandos `V`, `C` y `G` funcionan y que el contador avanza de uno en uno. Con un botón real los números exactos pueden variar, pero la tendencia es la misma.

Pruebas de seguridad (apagón) y de arranque

- **Prueba del apagón (RESET con el motor girando):** con el motor a velocidad máxima se oprimió RESET en la simulación (Figura 2). Al volver, `setup()` dejó el motor apagado, apareció el aviso "SISTEMA REINICIADO" en la pantalla y en el Monitor Serie, y el sistema quedó esperando una nueva pulsación. El motor solo arrancó al oprimir el botón de nuevo, y lo hizo con rampa.
- **Dedo en el botón durante el reinicio:** como `iniciar()` toma la lectura inicial como estado aceptado, un botón que sigue oprimido al reiniciar no cuenta como pulsación y el motor no arranca solo.

- **Arranque de golpe contra rampa:** al arrancar de golpe (PWM directo a 255) el motor pide de inmediato la corriente máxima de arranque, que es justo el pico que hace caer el voltaje de la fuente. Con la rampa, la velocidad sube poco a poco durante 1.5 s (en la simulación se ve cómo las rpm del motor aumentan gradualmente), la corriente crece de forma suave y el pico es mucho menor.

Observaciones sobre el comportamiento del sistema

- El filtro vive dentro de la clase `Boton`, por lo que se puede reutilizar con cualquier otro botón sin repetir código en el `loop()`.
- El contador solo suma al pasar de suelto a oprimido; soltar el botón no cuenta, aunque el rebote al soltar meta una bajada falsa a LOW, porque esa bajada dura menos que la ventana y se descarta.
- Para que el L298N interprete correctamente las señales de D7 y D9, su GND debe estar unido al GND del Arduino.
- El motor se alimenta siempre con una fuente separada y nunca desde el pin 5V del Arduino; el software ayuda a defenderse de los reinicios, pero no arregla un problema de alimentación.
- Con `millis()` y sin `delay()`, el programa sigue leyendo el botón, atendiendo el Monitor Serie, avanzando la rampa y actualizando la pantalla mientras el filtro espera.

5. Preguntas de reflexión

1. Explica con tus palabras qué es el rebote y por qué el Arduino lo detecta aunque tú no lo notes.
El rebote es que los contactos del botón no se quedan quietos al primer golpe: la laminita de metal choca, rebota varias veces y apenas después se queda pegada, igual que una pelota que cae al piso. Todo dura de 1 a 10 ms, así que una persona no lo percibe, pero el Arduino lee el pin millones de veces por segundo y ve la señal cambiar varias veces (oprimido, suelto, oprimido, suelto, oprimido), como si hubieran sido varias pulsaciones.

2. ¿Por qué el contador debe sumar solo al pasar de suelto a oprimido y no en cualquier cambio de la señal?
Porque una pulsación completa produce dos cambios de la señal, uno al oprimir y otro al soltar. Si se sumara en cualquier cambio, cada pulsación contaría 2 aunque no hubiera rebote. Al contar solo el paso de HIGH a LOW (suelto a oprimido), oprimir y soltar suma exactamente 1.

3. ¿Por qué el rebote al soltar el botón también puede sumar pulsaciones?
Porque al soltar los contactos también rebotan: la señal pasa por HIGH, LOW, HIGH. Aunque la persona ya está soltando, ese rebote mete una bajada falsa a LOW y un programa sin filtro la cuenta como si se hubiera oprimido otra vez. El filtro la descarta porque esa bajada dura mucho menos que la ventana de 25 ms.

4. ¿Qué problema tendría usar delay(50) para filtrar el rebote en lugar de millis()?
Que `delay(50)` detiene todo el programa durante 50 ms. En ese tiempo el Arduino no atiende el Monitor Serie, no avanza la rampa del motor ni actualiza la pantalla, así que el sistema se vuelve torpe y deja de responder. Además, una sola lectura después de la pausa puede caer todavía en un rebote. Con `millis()` el programa solo compara tiempos y sigue funcionando mientras espera a que el botón se calme.

5. Según tus pruebas, ¿qué pasa si la ventana de estabilidad es muy corta? ¿Y si es muy larga?
Si la ventana es muy corta (por ejemplo 2 ms), es menor que lo que duran los rebotes, por lo que algunos se cuelan y se cuentan como pulsaciones de más (en la simulación contó 11 de 10). Si es muy larga (por ejemplo 300 ms), los rebotes se filtran, pero el botón se siente lento y se pierden las pulsaciones rápidas, porque el sistema espera demasiado para aceptar el cambio (contó solo 5 de 10). Una ventana de 25 ms es un buen equilibrio: es mayor que los rebotes y no se nota al usar el botón.

6. ¿Por qué es peligroso que un sistema reinicie con el motor encendido? Da un ejemplo industrial.
Porque al volver del reinicio el programa empieza desde cero y, si no se defiende, puede hacer algo que nadie ordenó, como arrancar el motor por sí solo. Un ejemplo industrial es una banda transportadora o una prensa: si el control se reinicia por una caída de voltaje mientras un operador la está limpiando o revisando, la máquina podría arrancar sola y lastimarlo. Por eso el sistema debe volver siempre apagado y esperar una orden nueva.

7. ¿Por qué la rampa de arranque ayuda a evitar caídas de voltaje?
Porque un motor quieto pide mucha más corriente al arrancar que cuando ya está girando. Si el PWM pasa de 0 a 255 de golpe, ese pico de corriente hace caer el voltaje de la fuente y puede reiniciar al Arduino. Con la rampa la velocidad sube poco a poco, la corriente aumenta de forma gradual y el pico es mucho menor, así que es menos probable que el voltaje baje del mínimo.

8. Menciona dos medidas de hardware que ayudan a evitar caídas de voltaje y que el software no puede reemplazar.
La primera es alimentar el motor con una fuente separada y con corriente suficiente (las pilas 18650 en lugar del pin 5V del Arduino), con las tierras unidas, para que el arranque del motor no jale el voltaje de la lógica. La segunda es usar cables gruesos y conexiones firmes, y colocar capacitores de filtrado cerca de la alimentación del motor, para absorber los picos de corriente y evitar malos contactos que provoquen caídas.

6. Conclusiones individuales

Ximena
En esta práctica entendí que un botón no es tan simple como parece: al oprimirlo, el Arduino puede ver varias pulsaciones por culpa del rebote. Resolverlo con una ventana de 25 ms medida con millis() hace que cada pulsación cuente una sola vez y que el programa nunca se bloquee, algo que no se logra con delay().

Carlos Tadeo
Lo más importante que vi fue la diferencia entre contar cualquier cambio de la señal y contar solo el paso de suelto a oprimido. También entendí por qué el sistema debe arrancar siempre en estado seguro: después de un reinicio el motor no debe arrancar solo, y la rampa de arranque ayuda a que el motor no cause las caídas de voltaje que reinician la placa.

Dulce Princesa
Con esta práctica vi cómo encapsular el filtro en la clase Boton hace el programa más ordenado y reutilizable: el loop solo pregunta si el botón fue presionado. También aprendí que el software ayuda pero no lo arregla todo, y que la alimentación del motor debe ir separada, con tierra común, para evitar los apagones desde el hardware.